

1. Number of Unique States

A “state” is defined by the unique inputs passed to your function $f(s, t)$.

- The string s will always be a contiguous substring of the original string S .
- The string t will always be a contiguous substring of the original string T .
- Your base case `if(s.length() != t.length())` ensures that we only process pairs where both substrings have the exact same length, let's call it L .

For any given length L (ranging from 1 to n):

- There are $(n - L + 1)$ possible starting positions for substring s in S .
- There are $(n - L + 1)$ possible starting positions for substring t in T .

So, for a specific length L , there are $(n - L + 1) \times (n - L + 1) = (n - L + 1)^2$ possible unique pairs of (s, t) .

To find the total number of states across all possible lengths, we sum this up:

$$\text{Total States} = \sum_{L=1}^n (n - L + 1)^2$$

Using sum of squares formulas, this evaluates to roughly $\frac{n^3}{3}$. Therefore, the **Total Number of Unique States** is $\mathcal{O}(n^3)$.

2. Work Done Per State

Now, let's look at how much time it takes to process one specific state (s, t) of length L , assuming its subproblems are already solved and cached:

1. **String Concatenation:** Creating `string key = s + "$" + t`; takes $\mathcal{O}(L)$ time because it has to copy the characters.
2. **The For Loop:** The loop runs from $c = 1$ to $L - 1$. So it iterates $\mathcal{O}(L)$ times.
3. **Inside the Loop (substr):** In each iteration, you are using `.substr()` to slice strings. Creating these new strings takes time proportional to their length, which is $\mathcal{O}(L)$.

Because you do $\mathcal{O}(L)$ work (the `substr` calls) inside a loop that runs $\mathcal{O}(L)$ times, the total work per state is $\mathcal{O}(L) \times \mathcal{O}(L) = \mathcal{O}(L^2)$.

3. Total Time Complexity

To get the final Time Complexity, we multiply the number of states of length L by the work done for a state of length L , and sum it up over all lengths:

$$\text{Total Time} = \sum_{L=1}^n [(n - L + 1)^2 \times \mathcal{O}(L^2)]$$

Expanding this polynomial gives terms with highest degree $n^2 \cdot L^2$, and integrating/summing L^2 up to n bumps the highest power to n^5 .

- **Final Time Complexity:** $\mathcal{O}(n^5)$

4. Space Complexity

- **Recursion Stack:** The maximum depth of the recursive call stack is n .
 - **Memoization Table:** The `unordered_map` stores at most $\mathcal{O}(n^3)$ unique keys. Each key is a string of length $2L + 1$, which takes up to $\mathcal{O}(n)$ space. Therefore, the memory used by the keys in the map is $\mathcal{O}(n^3) \times \mathcal{O}(n) = \mathcal{O}(n^4)$.
 - **Final Space Complexity:** $\mathcal{O}(n^4)$
-